

Año 2025 – 2026

Universidad : Universidad Loyola

Curso : Máster en Inteligencia Artificial

Profesor(s) : Bernardo Ronquillo Japón



MLOps & AI BI

LAB 6: Despliegue con Docker: Informe de reflexión

Ignacio González

4 de abril de 2026

Resumen

En este informe de reflexión se abordan conceptos fundamentales sobre el despliegue de aplicaciones de Machine Learning utilizando Docker. Se analizan aspectos críticos de configuración de red, como la exposición de puertos mediante `-host 0.0.0.0` frente a la interfaz *loopback*, y la comunicación interna entre contenedores en una red *bridge* gestionada por Docker Compose. Además, se explora la optimización en la construcción de imágenes a través de la caché de capas y las mejores prácticas para la gestión de secretos.

Finalmente, se presenta la solución a un problema práctico de concurrencia durante el despliegue: la sincronización del arranque de servicios mediante *Healthchecks*. Esto garantiza que el frontend no reciba tráfico hasta que el modelo de Machine Learning del backend esté completamente cargado en memoria, asegurando una experiencia de usuario estable y sin errores HTTP 503.

Términos : Docker, Docker Compose, Healthcheck, Uvicorn, MLOps, Redes

Índice

1 Preguntas de reflexión	3
1.1 Pregunta 1: ¿Por qué usar <code>-host 0.0.0.0</code> en lugar de <code>127.0.0.1</code> ?	3
1.2 Pregunta 2: ¿Por qué <code>localhost:8000</code> no funciona para comunicación entre contenedores?	3
1.3 Pregunta 3: Caché de capas Docker y orden de instrucciones	3
1.4 Pregunta 4: Alternativas para la gestión de secretos	3
2 Ejercicio de programación: HEALTHCHECK y arranque ordenado	4
2.1 Problema a resolver	4
2.2 Solución implementada	4
2.3 Flujo temporal	5

1 Preguntas de reflexión

1.1 Pregunta 1: ¿Por qué usar `-host 0.0.0.0` en lugar de `127.0.0.1`?

`127.0.0.1` es la dirección de *loopback*: el servidor solo acepta conexiones originadas desde el propio proceso, es decir, desde dentro del mismo contenedor. Docker realiza *port-forwarding* a través de la interfaz de red virtual del contenedor, no a través de *loopback*. Por tanto, si Uvicorn escucha solo en `127.0.0.1`, las peticiones llegadas por el *port-mapping* del host (`-p 8000:8000`) son descartadas y el servicio es inaccesible desde fuera.

Con `-host 0.0.0.0` el servidor escucha en todas las interfaces de red del contenedor (*loopback* + red virtual de Docker), permitiendo que el host y otros contenedores se comuniquen con él.

1.2 Pregunta 2: ¿Por qué `localhost:8000` no funciona para comunicación entre contenedores?

Cada contenedor tiene su propio *namespace* de red aislado. La interfaz `localhost` (*loopback*) de un contenedor apunta al propio contenedor, no a ningún otro. Cuando el frontend intenta conectar a `localhost:8000`, está buscando el puerto 8000 en sí mismo (en el contenedor del frontend), donde no hay ningún servidor.

Docker Compose crea una red *bridge* interna y registra automáticamente entradas DNS para cada servicio por su nombre (`backend`, `frontend`). Por eso la URL correcta para la comunicación inter-contenedor es `http://backend:8000`: el motor DNS interno de Docker resuelve `backend` a la IP del contenedor del backend dentro de la red `iris-network`.

1.3 Pregunta 3: Caché de capas Docker y orden de instrucciones

Docker construye imágenes en capas: cada instrucción del `Dockerfile` genera una capa que se cachea. Si una capa cambia, todas las capas posteriores se invalidan y deben reconstruirse.

En nuestros Dockerfiles seguimos el patrón recomendado:

```
COPY requirements.txt .          # Capa 1: solo el fichero de dependencias
RUN pip install --no-cache-dir   # Capa 2: instalación (lenta, ~minutos)
COPY . .                        # Capa 3: código de la aplicación
```

Gracias a este orden, cuando únicamente modificamos el código Python (`main.py`, `gradio_app.py`, etc.) Docker reutiliza las capas 1 y 2 de la caché y solo reconstruye la capa 3. Si invirtiéramos el orden (`COPY . .` antes de `RUN pip install`), cualquier cambio en el código invalidaría la caché de la instalación de dependencias, multiplicando el tiempo de *build* innecesariamente.

1.4 Pregunta 4: Alternativas para la gestión de secretos

Para este laboratorio usamos el fichero `.env` (conveniente para desarrollo local). En producción la opción recomendada son los servicios externos, ya que permiten rotar el token sin redespargar la aplicación y registran cada acceso al secreto.

Cuadro 1 – Alternativas para la gestión de secretos.

Método	Descripción	Riesgo
.env file	Variable en fichero .env leído por Docker Compose	Bajo si .env está en .gitignore; alto si se sube al repo por error
Docker Secrets	Secretos montados en memoria (/run/secrets/) en modo Swarm	Más seguro: nunca en variables de entorno ni en capas de imagen
Servicios externos	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault	El más seguro: rotación automática, auditoría, control de acceso granular

2 Ejercicio de programación: HEALTHCHECK y arranque ordenado

2.1 Problema a resolver

El backend carga el modelo desde Hugging Face Hub en un hilo secundario al arrancar (`load_model_background`). El contenedor está activo desde el primer momento, pero el modelo puede tardar 10-30 segundos en estar en memoria. Sin coordinación, el frontend arranca inmediatamente, llama a `/predict` y recibe HTTP 503, mostrando errores al usuario.

2.2 Solución implementada

1. Endpoint `/health` con semántica HTTP correcta:

```
if model_state["loaded"]:
    return {"status": "ok", ...} # HTTP 200 -> modelo listo
raise HTTPException(status_code=503, ...) # HTTP 503 -> cargando
```

2. HEALTHCHECK en el Dockerfile del backend:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
  CMD python -c "import urllib.request; \
  urllib.request.urlopen('http://localhost:8000/health')" || exit 1
```

- `start-period=60s`: periodo de gracia; Docker no cuenta fallos durante los primeros 60 s mientras el modelo se descarga.
- `urllib.request.urlopen` lanza `HTTPError` para respuestas 503 → `exit 1` → estado **unhealthy**. Para respuestas 200 → `exit 0` → **healthy**.
- Se usa `urllib` (stdlib) porque `python:3.11-slim` no incluye `curl`.

3. condition: `service_healthy` en `docker-compose.yml`:

```
depends_on:
  backend:
    condition: service_healthy
```

El orquestador retiene el contenedor del frontend hasta que el backend supera el *health check* (estado *healthy*). De este modo, cuando Gradio arranca, el modelo ya está cargado y `/predict` responde correctamente desde el primer momento.

2.3 Flujo temporal

t=0s backend arranca, uvicorn responde en /health con 503
 Docker HEALTHCHECK: unhealthy (dentro de start_period -> no cuenta)

t=~20s modelo descargado desde HF Hub, model_state["loaded"]=True
 /health devuelve 200 -> Docker marca backend como healthy
 Docker Compose lanza el contenedor del frontend

t=~25s frontend listo, Gradio accesible en <http://localhost:7860>
 Primera predicción exitosa

